



Streamlit



Agenda

- What is Streamlit
- Use Cases
- How to Start
- Demo

What is Streamlit?



Streamlit is a Python library
for building data apps



Streamlit is a Python library
for building data apps
blazing fast



Streamlit is a Python library
for building data apps
blazing fast

It's an app framework



Streamlit is a Python library
for building data apps
blazing fast

It's an app framework

*It's not meant for
general-purpose apps.
It focuses on "data apps"*



Streamlit is a Python library
for building data apps

blazing fast

It's an app framework

*It optimizes for time-to-insight
and developer experience*

*It's not meant for
general-purpose apps.
It focuses on "data apps"*



Why is this important?



The world has changed

The world has changed

Analytics

The world has changed

Analytics → **Data science and ML**

The world has changed

Analytics → **Data science and ML**

Real data is messy
and demand a lot more interpretation
than simple filters can provide.



The world has changed

Analytics → **Data science and ML**

Reports



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

*Data consumers need interactivity
so they can try
new things on the fly*



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL → **Python**



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL → **Python**

*Python brings the power of
a world of libraries to build upon.
It's the language
of data science and ML*



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL → **Python**

No code



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL → **Python**

No code → **Yes code!**



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL → **Python**

No code → **Yes code!**

Real code shines at collaboration and reuse.

It integrates with everything:

Git, code reviews, testing, CI/CD, etc.



The world has changed

Analytics → **Data science and ML**

Reports → **Apps**

SQL → **Python**

No code → **Yes code!**

Real code shines at collaboration and reuse.

It integrates with everything:

Git, code reviews, testing, CI/CD, etc.

And it's the language of LLMs!



So it's Python, apps, yes code.



But apps are hard!



73%

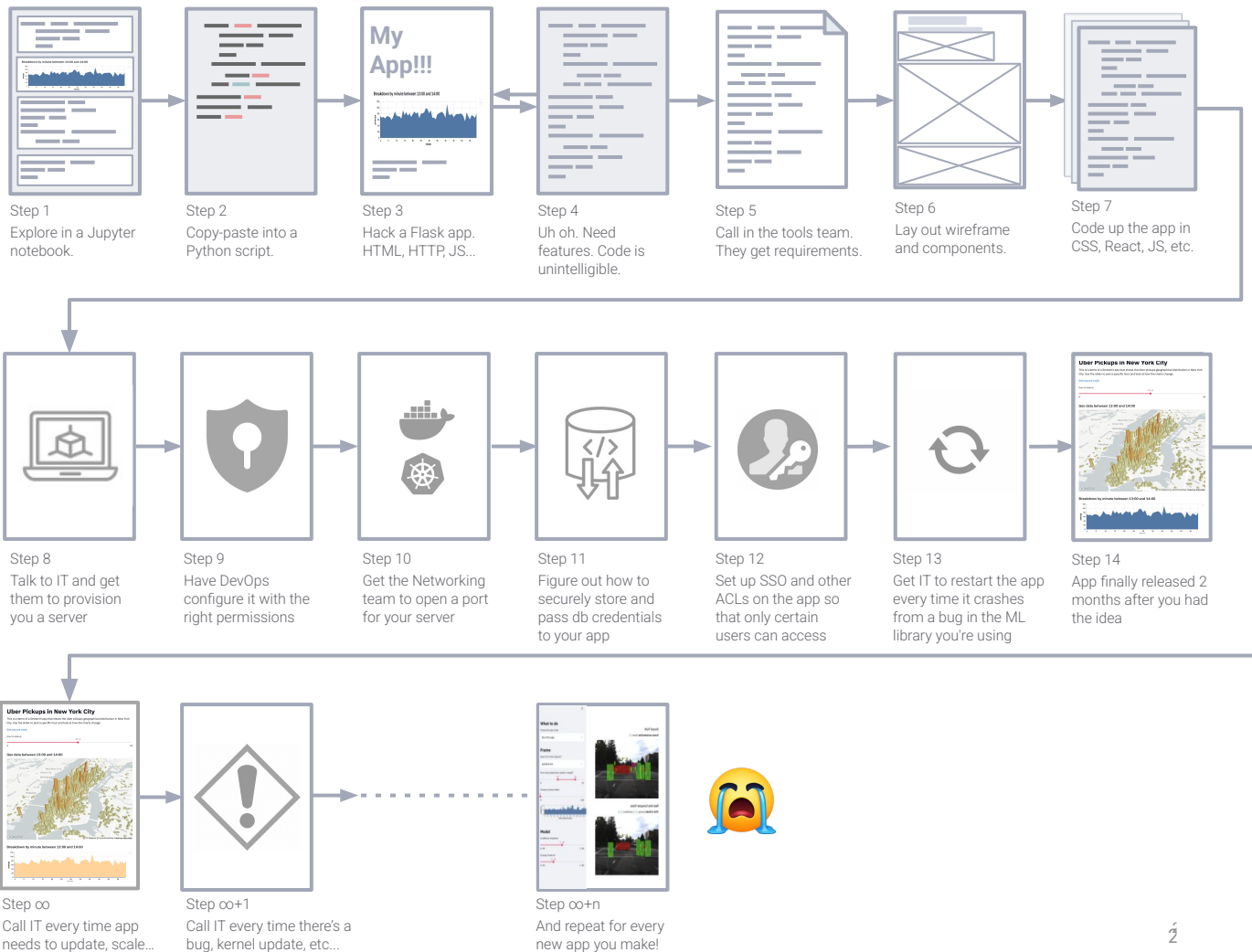
of data is not rendered effectively because it's just too hard to build the tools to use it

[Source](#)



Getting from data to action is hard

Only a fraction of data questions are ever answered since companies haven't built tools to capture or analyze the data. 73% of data is rendered effectively unused* because it's just too hard to build the tools to use it.



MVC

NPM

Webpack

CSS

JS

React

D3

Node

HTML

You're not a frontend engineer,



MVC *NPM* *Webpack*
HTTP *CSS* *JS* *React* *D3* *Node*
HTML *Deserialization*
Routing *Serialization*
Templates *Flask* *Caddy*
Error codes *Auth*

You're not a frontend engineer,
backend engineer,



MVC *NPM* *Webpack*
HTTP *CSS* *JS* *React* *D3* *Node*
HTML *Deserialization*
Routing *Serialization*
Templates *Autodeploy* *Flask* *Caddy*
Error codes *K8S* *Monitoring* *Auth*
Rate-limiting *Rate-limiting*

You're not a frontend engineer,
backend engineer,
devops engineer,



You're not a frontend engineer,
backend engineer,
devops engineer,
designer

MVC

NPM

Webpack

JS

React

D3

Node

Deserialization

HTTP

CSS

HTML

Serialization

Routing

Templates

Autodeploy

Flask

Caddy

Error codes

K8S

Auth

Rate-limiting

Monitoring

Rate-limiting

White space

Design systems

Grid

Borders, shadows

Alignment

Font families



You're not a frontend engineer,
backend engineer,
devops engineer,
designer

**and it's not a good use of your time
to impersonate those!**

White space
Design systems
Borders, shadows

THIS

x1000

OMGYES

+1 +1
+1

Grid
Alignment
Rate-limiting
Font families

Auth
Flask
Autodeploy
Monitoring
Caddy

Serialization

Deserialization

Webpack

Node

D3

React

JS

NPM

CSS

MVC

HTML

HTTP

Routing

Templates

Error codes

Rate-limiting

K8S

White space

Design systems

Borders, shadows



Streamlit works on 3 core principles

(1)

Embrace Python
scripting

(2)

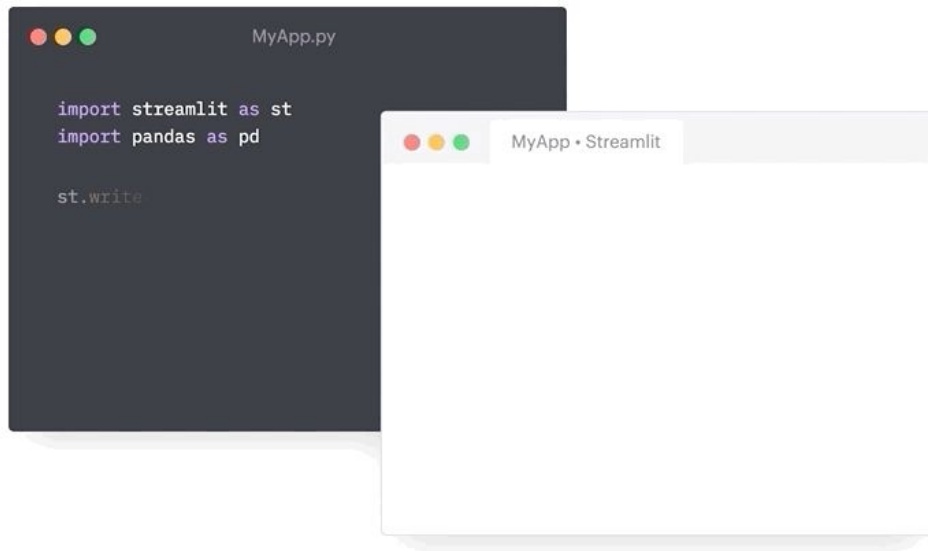
Treat widgets as
variables

(3)

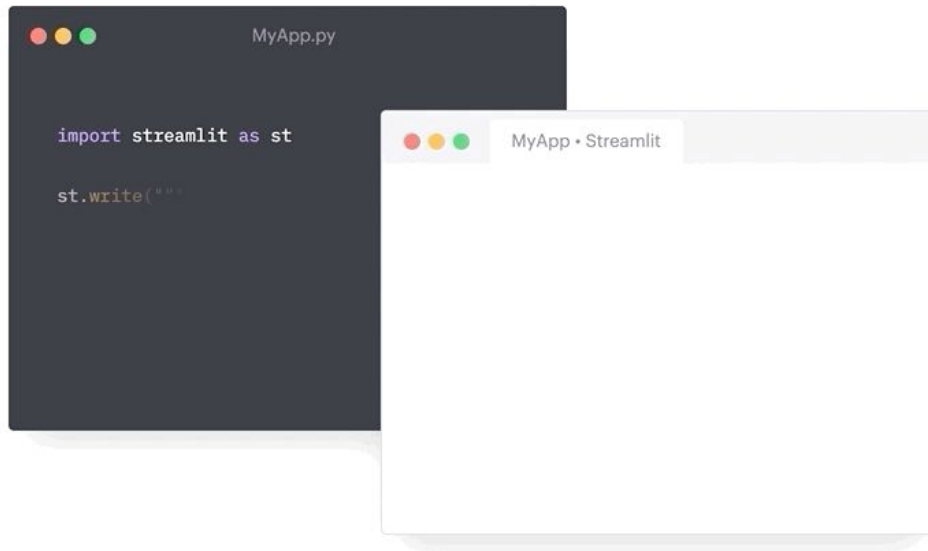
Reuse data and
computation



1. Embrace Python scripting

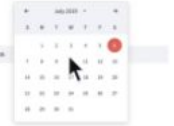
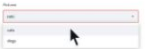


2. Treat **widgets** as variables



Streamlit API

<https://docs.streamlit.io/develop/api-reference>

 Button Display a button widget. <pre>clicked = st.button("Click me")</pre>	 Download button Display a download button widget. <pre>st.download_button("Download", "Download")</pre>	 Checkbox Display a checkbox widget. <pre>selected = st.checkbox("I agree")</pre>	 Number input Display a numeric input widget. <pre>choice = st.number_input("Pick a number")</pre>	 Text-area Display a multi-line text input widget. <pre>text = st.text_area("Text to input")</pre>	 Date input Display a date input widget. <pre>date = st.date_input("Your birthday")</pre>	 Progress bar Display a progress bar. <pre>for i in range(101): st.progress(i) do_something_slow()</pre>	 Spinner Temporarily displays a message while executing a block of code. <pre>with st.spinner("Please wait..."): do_something_slow()</pre>	 Balloons Display celebratory balloons! <pre>do_something() # Celebrate when all done! st.balloons()</pre>
 Radio Display a radio button widget. <pre>choice = st.radio("Pick one", "Dog", "Cat", "Goldfish")</pre>	 Selectbox Display a select widget. <pre>choice = st.selectbox("Pick a color", "red", "green", "blue", "yellow")</pre>	 Multiselect Display a multiselect widget. The multiselect widget starts as empty. <pre>choices = st.multiselect("Buy", "apples", "bananas", "oranges")</pre>	 Time input Display a time input widget. <pre>time = st.time_input("Meeting time")</pre>	 File Uploader Display a file uploader widget. <pre>data = st.file_uploader("Upload a file")</pre>	 Camera input Display a widget that allows users to upload images directly a camera. <pre>image = st.camera_input("Take a photo")</pre>	 Error box Display error message. <pre>st.error("We encountered an error")</pre>	 Warning box Display warning message. <pre>st.warning("Unable to fetch data")</pre>	 Info box Display an informational message. <pre>st.info("Dataset is updated")</pre>
 Slider Display a slider widget. <pre>number = st.slider("Pick a number", 0, 100)</pre>	 Select-slider Display a slider widget to select items from a list. <pre>size = st.select_slider("Pick a size", ["small", "medium", "large"])</pre>	 Text input Display a single-line text input widget. <pre>name = st.text_input("First name")</pre>				 Success box Display a success message. <pre>st.success("Match found!")</pre>	 Exception output Display an exception. <pre>e = RuntimeError("This is an exception") st.exception(e)</pre>	



3. Reuse data and computation

Instantiate DB/model
connectors only once

```
@st.cache_resource
def get_connector():
    connector = st.connection("snowflake")
    return connector

@st.cache_data(ttl=600)
def query_database(_connector):
    df = _connector.query("SELECT * FROM users")
    return df
```

Store query results in cache

Add in secrets!



Streamlit lets you go straight from Python script to web app

```
import streamlit as st
import pandas as pd
import altair as alt

@st.cache
def get_UN_data():
    AWS_BUCKET_URL = "https://streamlit-demo-data.s3-us-west-2.amazonaws.com"
    df = pd.read_csv(AWS_BUCKET_URL + "/agri.csv.gz")
    return df.set_index("Region")

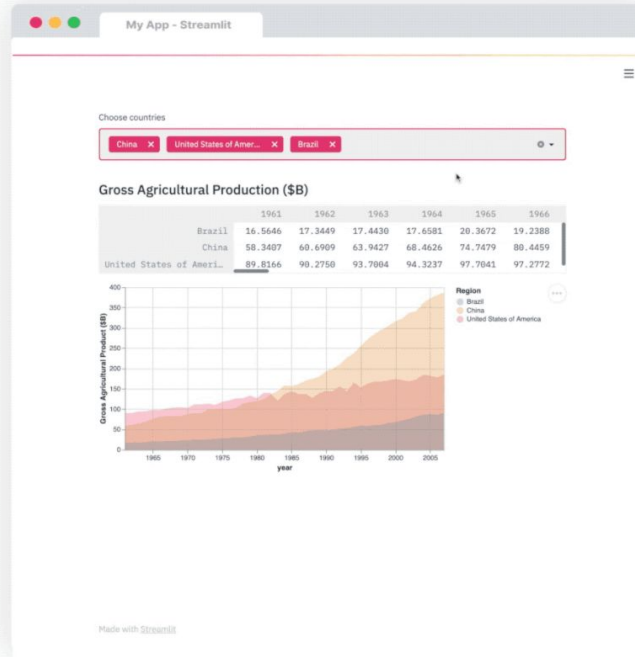
df = get_UN_data()

countries = st.multiselect(
    "Choose countries", list(df.index), ["China", "United States of America"]
)

data = df.loc[countries]
data /= 1000000.0
st.write("## Gross Agricultural Production ($B)", data.sort_index())

data = data.T.reset_index()
data = pd.melt(data, id_vars=["index"], rename(
    columns={"index": "year", "value": "Gross Agricultural Product ($B)"
})
chart = (
    alt.Chart(data)
    .mark_area(opacity=0.3)
    .encode(
        x="year:T",
        y=alt.Y("Gross Agricultural Product ($B):Q", stack=None),
        color="Region:N",
    )
)

st.altair_chart(chart, use_container_width=True)
```

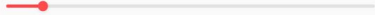


From code to app in minutes!



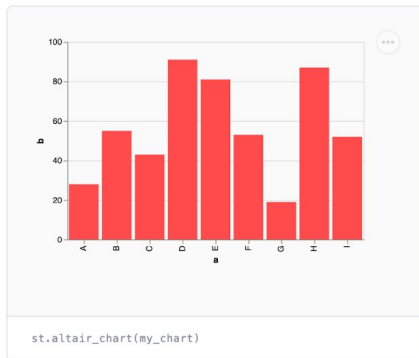
It's easy to add in interaction, layout, and themes

Pick a number



0 100

```
number = st.slider("Pick a number", 0, 100)
```



Pick a file

Drag and drop files here
Limit 200MB per file • TXT

Browse files

```
file = st.file_uploader("Pick a file")
```

Pick a color



```
color = st.color_picker("Pick a color")
```

Pick a pet

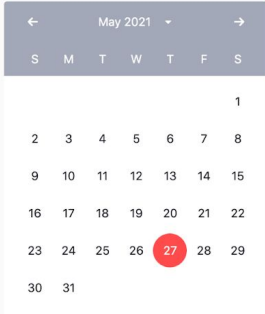
☒ Dog

☐ Cat

☐ Bird

```
pet = st.radio("Pick a pet", ["Dog", "Cat", "Bird"])
```

Pick a date



```
date = st.date_input("Pick a date")
```

Choose from light
or dark theme or
set your own
custom colors



And it works with all your favorite Python libraries

bokeh

Altair

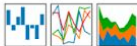
PyTorch

OpenCV

DECK.GL

pandas

$y_i = \beta^T x_i + \mu_i + \epsilon_i$



Vega-Lite

matplotlib

NumPy

learn

TensorFlow

plotly

K Keras

And many more!

<https://repo.anaconda.com/pkg/snowflake/>



USE CASES



Common use cases for Streamlit

Doing Data Science

Streamlit is primarily targeted at a data science audience, though many ML eng, Data Eng, and others use it as well. Look for companies already using Snowflake's Data Science workflows.

Coding in Python

If they are using Snowpark for Python that's a good signal! If they are not using Python already, then they are probably better off (at the moment) using their existing workflows.

Looking for speed!

There are many options for making data apps. Snowflake supports and will continue to support lots of options. You don't have to remake apps in Streamlit, but it's a good place to start if you're in Python and want something fast.



Common use cases for Streamlit

Rapid prototyping

Data science teams quickly build data apps in Python to test hypotheses, gather feedback, and iterate quickly to deliver insights to their business stakeholders.

Demo-ing work

Data scientists use Streamlit as a direct method of sharing data with clients, customers, partners, and even suppliers to reduce lag and drive positive business outcomes. Often this starts as just a demo of capabilities but then turns into deliverables for customers and partners.

Creating tools for business users

Data teams struggle to share the outputs of their models in an easily consumable way for their stakeholders. Folks from sales, operations, marketing, and support can often benefit from interacting with models the data team has built. Streamlit creates a fast and easy way to create new tools from data that empower business users.

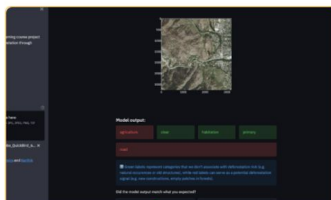


What Streamlit Can Do?



Monitoring apps

Tools that display trends and real-time insights.



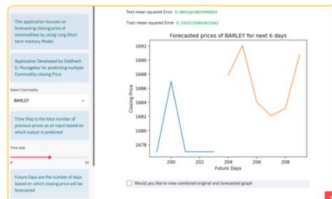
Analysis tools

Apps that use machine learning to analyze new data types.



Detection apps

Tools that use computer vision to detect and track objects.



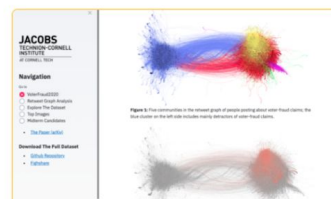
Prediction tools

Apps that predict prices, stock shortages, quality issues, etc.



Explanatory apps

Apps that analyze large datasets and present easy to understand insights.

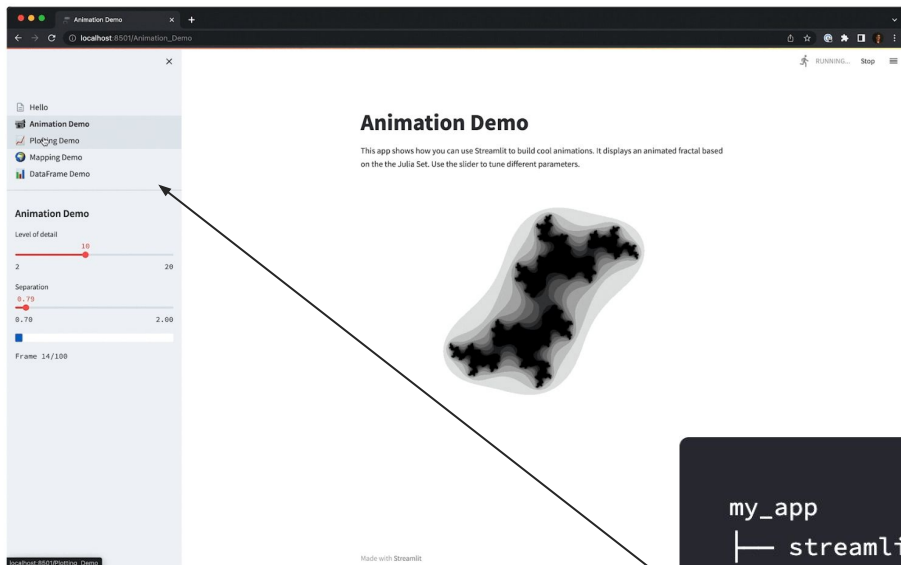


Interactive tools

Tools that allow you to interact with the data to gain new insights.



What Streamlit Can Do?



Native multi-page apps

```
my_app
├── streamlit_app.py  <-- Your main script
└── pages
    ├── page_2.py    <-- New page 2!
    └── page_3.py    <-- New page 3!
```



How to Start



Two Ways to Use Streamlit with Snowflake

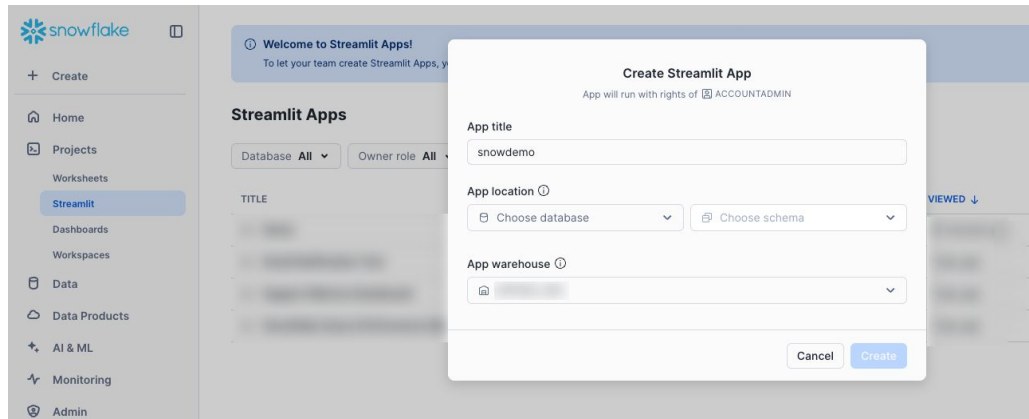
- Streamlit in your Python programs connected to Snowflake using Snowflake connector locally
- Streamlit in Snowflake



Remember, it only takes minutes to get started!

```
$ pip install streamlit  
$ streamlit hello
```

See our [docs](#) for more getting started info with the Streamlit open-source library



Streamlit Open Source Python Library

Bring your data to life in minutes by building an interactive app – all in Python

Get started today

- Install Anaconda (optional)
- Install streamlit
 - conda install or pip install streamlit
- Connect to Snowflake with Snowflake's Python connector
- Launch Visual Studio Code or IDE of your choice
- Start coding

```
import streamlit as st
import pandas as pd
import altair as alt

@st.cache
def get_us_data():
    AWS_BUCKET_URL = "https://streamlit-demo-data.s3-us-west-2.amazonaws.com"
    df = pd.read_csv(f"{AWS_BUCKET_URL}/agri.csv.gz")
    return df.set_index("Region")

df = get_us_data()

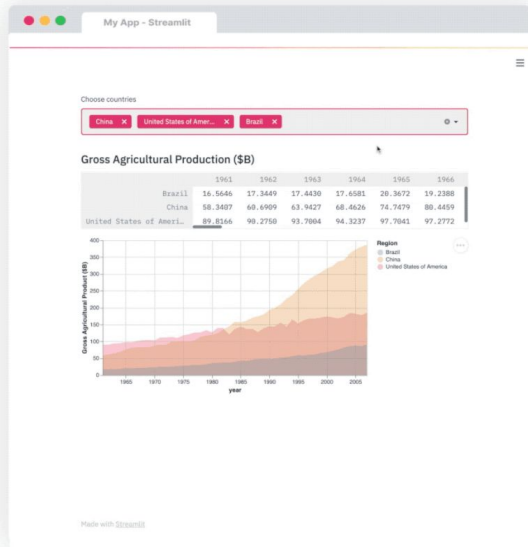
countries = st.multiselect(
    "Choose countries", list(df.index), ["China", "United States of America"]
)

data = df.loc[countries]
data /= 1000000.0
st.write("## Gross Agricultural Production ($B)", data.sort_index())

data = data.T.reset_index()
data = pd.melt(data, id_vars=["index"], rename(
    columns={"index": "year", "value": "Gross Agricultural Product ($B)"})
)

chart = (
    alt.Chart(data)
    .mark_area(opacity=0.3)
    .encode(
        x="year:T",
        yalt="Gross Agricultural Product ($B)Q", stackName(),
        color="Region:N",
    )
)

st.altair_chart(chart, use_container_width=True)
```



Create a Streamlit App in Snowsight

Streamlit Apps Demo

Active Share Run

Files Databases

Connect Git Repos...

environment.yml streamlit_app.py

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

Import python packages

import streamlit as st

from snowflake.snowpark.context import get_active_session

Write directly to the app

st.title(f"Example Streamlit App :balloon: {st.__version__}")

st.write(

"""Replace this example with your own code!

And if you're new to Streamlit, check

out our easy-to-follow guides at

docs.streamlit.io.

"""

)

Get the current credentials

session = get_active_session()

Use an interactive slider to get user input

hifives_val = st.slider(

"Number of high-fives in Q3",

min_value=0,

max_value=90,

value=60,

help="Use this to enter the number of high-fives you gave in Q3",

)

Create an example dataframe

Note: this is just some dummy data, but you can easily connect to your Snowflake database

It is also possible to query data using raw SQL using session.sql()

created_dataframe = session.create_dataframe(

[(50, 25, "Q1"), (20, 35, "Q2"), (hifives_val, 30, "Q3")],

schema=["HIGH_FIVES", "FIST_BUMPS", "QUARTER"],

)

Execute the query and convert it into a Pandas dataframe

queried_data = created_dataframe.to_pandas()

Create a simple bar chart

See docs.streamlit.io for more types of charts

st.subheader("Number of high-fives")

st.bar_chart(data=queried_data, x="QUARTER", y="HIGH_FIVES")

st.subheader("Underlying data")

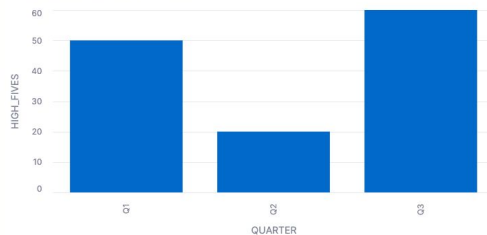
st.dataframe(queried_data, use_container_width=True)

Example Streamlit App 1.48.0

Replace this example with your own code! And if you're new to Streamlit, check out our easy-to-follow guides at docs.streamlit.io.

Number of high-fives in Q3

Number of high-fives



Underlying data

	HIGH_FIVES	FIST_BUMPS	QUARTER
0	50	25	Q1
1	20	35	Q2
2	60	30	Q3



Worksheets now support Python



Python runs under Anaconda



Benefit from Snowpark Python



Side-by-side editor in your browser



Share a Streamlit App with your colleagues

The screenshot shows the Streamlit App interface. On the left, a code editor displays Python code for a Streamlit app. The code imports packages, sets a title, and creates a bar chart. A sharing modal is open in the center, showing the 'Add role to share with' dropdown menu. The modal lists roles: ACCOUNTADMIN, TEST-ROLE, and View only. The 'View only' role is selected. The modal also shows the 'View and share' option. On the right, the app output is displayed, showing a bar chart titled 'Number of high-fives' and a table titled 'Underlying data'.

```
1 # Import python packages
2 import streamlit as st
3 from snowflake.snowpark.functions import col, lit
4
5 # Write directly to the app
6 st.title("Example Streamlit App")
7
8 """Replace this example with your own code!
9 **And if you're new to Streamlit, check out our
10 easy-to-follow guides at docs.streamlit.io!"""
11
12 # Get the current credentials
13 session = get_active_session()
14
15 # Use an interactive slider to get a value
16 hifives_val = st.slider(
17     "Number of high-fives in Q3",
18     min_value=0,
19     max_value=90,
20     value=60,
21     help="Use this to enter the number of high-fives you gave in Q3",
22 )
23
24 # Create an example dataframe
25 # Note: this is just some dummy data, but you can easily connect to your
26 # database and query data using raw SQL using session.sql()
27 created_dataframe = session.create_dataframe(
28     [[50, 25, "Q1"], [20, 35, "Q2"], [hifives_val, 30, "Q3"]],
29     schema=["HIGH_FIVES", "FIST_BUMPS", "QUARTER"],
30 )
31
32 # Execute the query and convert it into a Pandas dataframe
33 queried_data = created_dataframe.to_pandas()
34
35 # Create a simple bar chart
36 # See docs.streamlit.io for more types of charts
37 st.subheader("Number of high-fives")
38 st.bar_chart(data=queried_data, x="QUARTER", y="HIGH_FIVES")
39
40 st.subheader("Underlying data")
41 st.dataframe(queried_data, use_container_width=True)
```

Number of high-fives

HIGH_FIVES	FIST_BUMPS	QUARTER
50	25	Q1
20	35	Q2
71	30	Q3

Copy the link to share your app
Support for RBAC



STREAMLIT DEMO



```
import streamlit as st
import pandas as pd
```

```
st.write("""
# My first app
""")
```

```
num = st.slider("Pick a number", 1, 100)
st.write("The number is", num)
```

```
data = np.random.randn(num, 5)
```

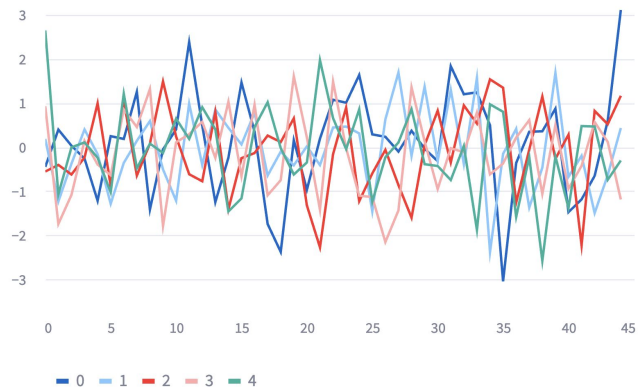
```
st.line_chart(data)
st.write(data)
```

My first app

Pick a number



The number is 45



0

1

2

3

4

APPENDIX - REFERENCES

Title	Description	Link
SIS Official Doc	Official SIS documentation	SIS Official Doc
Snowflake Streamlit Demo	Official Streamlit Github Repositories	Snowflake Streamlit Demo
Streamlit Gallery	Official Streamlit Gallery	Gallery • Streamlit
Getting Started with Snowpark for Python and Streamlit - Quickstart	Snowpark Python and Streamlit Quickstart	Getting Started With Snowpark for Python and Streamlit
SIS Deployment	Three ways to deploy SIS	Create and deploy Streamlit apps using Snowsight Create and deploy Streamlit apps using SQL Create and deploy Streamlit apps using Snowflake CLI



Questions or FAQ



Snowflake Partner 培训课程反馈问卷

